

Design and Analysis of Software Systems (DASS)

Complete Exam Study Guide

Spring 2026 — All Lectures Covered

Topics Covered:

- Introduction to Software Engg.
- Process Models
- Project Estimation
- Project Scheduling
- Use Case Modeling & UML
- Software Testing
- SDLC
- Agile & Scrum
- Requirements Engineering
- Project Tracking
- Conceptual Modeling
- Test Techniques & Metrics

IMPORTANT: This guide covers ALL lectures from 5-Jan through 21-Feb. Study every section carefully!

Contents

1	Introduction to Software Engineering	4
1.1	Why Software Engineering Matters	4
1.2	Green Field vs. Brown Field Systems	4
1.3	Five Pillars of Software Engineering (from Class)	4
1.4	Systems Thinking	5
2	Software Development Life Cycle (SDLC)	6
2.1	Phases of the SDLC	6
2.2	Detailed Phase Breakdown	7
2.3	The PDCA Cycle (Plan–Do–Check–Act)	7
2.4	Typical Time Distribution	7
3	Software Process Models	9
3.1	Waterfall Model	9
3.2	Prototyping Model	9
3.3	Evolutionary / Incremental Model	9
3.4	Spiral Model	10
3.5	Comparison of Process Models	10
3.6	The Cone of Uncertainty	10
4	Design Principles	11
4.1	SOLID Principles	11
4.2	Open-Closed Principle (OCP) — In Detail	11
4.3	GRASP Principles	12
4.4	Coupling & Cohesion	12
4.5	Types of Software Maintenance	12
5	Agile Development	14
5.1	The Agile Manifesto (2001)	14
5.2	12 Agile Principles (Key Ones)	14
5.3	Scrum Framework	14
5.4	User Stories	15
5.5	Key Agile Practices	15
5.6	DevOps	16
5.7	Agile Process Spectrum	16
6	Project Estimation & Scheduling	17
6.1	The Iron Triangle of Project Management	17
6.2	Function Point Analysis (FPA)	17
6.3	Value Adjustment Factor (VAF)	18
6.4	Work Breakdown Structure (WBS)	18
6.5	Gantt Charts & Critical Path	19
6.6	Earned Value Management (EVM)	20
7	Requirements Engineering	21
7.1	Functional vs. Non-Functional Requirements	21
7.2	Requirements Gathering Techniques	21
7.3	Qualities of Good Requirements	21
7.4	Categorizing Requirements: Basic, Differentiators, Delights	22

7.5	MoSCoW Prioritization	22
7.6	Minimum Viable Product (MVP)	22
7.7	Verification vs. Validation	22
7.8	Software Requirements Specification (SRS)	23
7.9	Data Privacy & Regulations	23
8	Use Case Modeling	24
8.1	Actors	24
8.2	Use Case Template	24
8.3	Use Case Relationships	25
8.4	Granularity of Use Cases	25
9	Conceptual Modeling & UML	26
9.1	From Use Cases to Conceptual Model	26
9.2	UML Class Diagram Notation	26
9.3	Access Modifiers	26
9.4	Associations & Multiplicity	27
9.5	Types of Attributes	27
9.6	Key Distinction: Conceptual vs. Software Classes	27
10	Software Testing	28
10.1	Levels of Testing	28
10.2	Integration Testing Strategies	29
10.3	Black Box vs. White Box Testing	29
10.4	Black Box Testing Techniques	29
10.4.1	Equivalence Partitioning	29
10.4.2	Boundary Value Analysis	29
10.4.3	State Transition Testing	30
10.5	White Box Testing Techniques	30
10.6	Test Case	30
10.7	Types of Acceptance Testing	30
10.8	Key Testing Principles	31
10.9	Testing Metrics	31
10.10	Qualities of a Good Tester	31
10.11	Regression Testing & Maintenance Testing	31
11	Exam Traps & Warnings (From Quiz 1 Answer Key)	32
12	Crossword Vocabulary — Key Terms & Definitions	33
12.1	Terms from Quiz 1 Crossword (All Sets)	33
12.2	Additional Likely Crossword Terms (New Topics Since Quiz 1)	33
13	Scenario-Based Practice Questions (Quiz 1 Style)	35
13.1	Practice Scenario 1: Hospital Management System	35
13.2	Practice Scenario 2: E-Commerce Platform	36
13.3	Practice: Use Case & Conceptual Modeling Question	36
13.4	Practice: Testing Question	37
14	Additional Practice Questions	39
14.1	Short Answer Questions	39
14.2	Comparison Questions	39
14.3	Multiple Choice Quick Review	40

15 Quick Reference Cheat Sheet

42

1 Introduction to Software Engineering

What is Software Engineering?

Software Engineering is the application of **engineering principles and techniques** to the **design, development, testing, and maintenance** of software systems. It is NOT just about writing code — it is about building **systems** that are reliable, maintainable, and scalable.

1.1 Why Software Engineering Matters

Key Insight from Class

A simple modern-day game like Candy Crush has **millions of lines of code**. The new Mahindra XEV 9e has **75 million lines of code!** Even at peak productivity (1 line/minute), one engineer can produce only **120,000 lines/year**. Building software is fundamentally about **teamwork**, not individual brilliance.

- **Industry Average:** If you can move **25 lines of code per day** to production, you are a brilliant programmer. 99% of what you write will NOT move to production directly.
- **People First, Not Technology First:** It is about what your code can do for others, and how you can sustain what you write by keeping it as simple as possible.
- **Maintenance dominates:** The development cycle takes $\approx$ **20%** of effort; maintenance takes $\approx$ **80%** of effort.
- Only **10–20%** of software projects are truly successful (on time, within budget, meeting specs).

1.2 Green Field vs. Brown Field Systems

Green Field vs. Brown Field

Type	Description
Green Field	A brand new software system developed from scratch. Very rare in the real world.
Brown Field	An existing software system that is modified, extended, or enhanced. This is what you will encounter 99% of the time in industry.

Remember!

You will almost NEVER start from scratch in the real world. You will always be building on top of existing systems. Understanding existing codebases and being able to extend them is a **critical skill**.

1.3 Five Pillars of Software Engineering (from Class)

The professor emphasized **five key skills** that every software engineer must have:

#	Skill
1	Design — Thinking before coding; systematic approach
2	Teamwork — Complementary skills; collaboration
3	Communication — With team, clients, stakeholders
4	Programming — Writing clean, maintainable code
5	Documentation — So others can understand and maintain your work

1.4 Systems Thinking

Beyond Software Engineering

Modern software is part of **larger systems**. The “S” in DASS stands for **Systems**. You must become a **systems engineer** — someone who understands the entire system (software + hardware + people + processes), not just the code.

Knight Capital Group Case Study (2012)

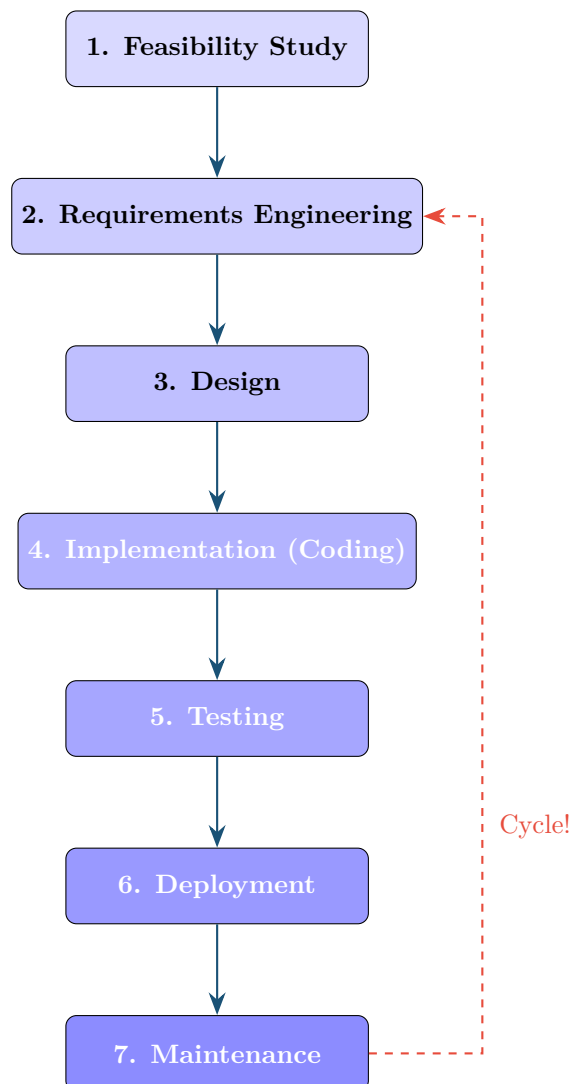
A small software update to a trading system caused a **45-minute outage** that resulted in **millions of dollars in losses**. The company had to be sold because they could not sustain the losses. **Lesson:** Proper process **MUST** be followed during updates and deployments.

2 Software Development Life Cycle (SDLC)

What is SDLC?

The **Software Development Life Cycle (SDLC)** is a systematic framework used to **plan, design, develop, test, and deliver** software applications. It divides the development process into multiple **phases**, each with defined activities and deliverables.

2.1 Phases of the SDLC



2.2 Detailed Phase Breakdown

Phase	Description & Key Activities
Feasibility Study	Evaluate whether the project is viable. Assess technical , economic , operational , and scheduling feasibility. Determine resources, risks, and potential benefits.
Requirements	Gather, analyze, and document what the software must do. Produce the Software Requirements Specification (SRS) document.
Design	Create the blueprint: architecture, components, interfaces. Translate requirements (problem space) into a solution (solution space).
Implementation	Write the actual code. Follow coding standards. Build what was designed.
Testing	Verify that the software meets requirements. Static analysis (linting) + Dynamic analysis (running tests).
Deployment	Release the software to the production environment. Consider the deployment environment carefully.
Maintenance	Update, modify, fix, and enhance the software. This is where 70–80% of the effort and money goes!

Why is it called a “Cycle”?

Because after Version 1.0, the client will want Version 2.0! You go back through the phases: new requirements → updated design → new code → more testing → new release. Software is **never constant** — it keeps evolving. Sub-cycles can occur too (e.g., going back from testing to design if issues are found).

2.3 The PDCA Cycle (Plan–Do–Check–Act)

PDCA / Deming Cycle

A framework for **continuous improvement**:

1. **Plan:** Identify goals and objectives; plan the approach.
2. **Do:** Execute the plan; implement the solution.
3. **Check:** Verify the results; compare with requirements.
4. **Act:** Implement improvements based on what was learned.

This cycle repeats continuously to drive ongoing improvement in the development process.

2.4 Typical Time Distribution

Activity	Approximate Time
Requirements & Design	20–30%
Implementation (Coding)	15–20%
Testing	30–40%
Maintenance	60–80% of total lifetime

Key Takeaway

Coding is only a small part of software engineering. **Understanding code, designing systems, and maintaining them** is far more important in the real world than just writing code fast.

3 Software Process Models

What is a Process Model?

A **process model** defines the **order and manner** in which SDLC phases are carried out. Different models suit different project types, risk levels, and requirements stability.

3.1 Waterfall Model

Waterfall Model (Traditional / Plan-Driven)

How it works: A **linear, sequential** approach. Each phase must be **completed fully** before moving to the next. No going back.

Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance

When to use:

- Requirements are **well-understood and stable**
- The project is **small to medium** in size
- Technology is **well-known**

Strengths: Simple, easy to manage, clear milestones.

Weaknesses: Inflexible; no working software until late; risky if requirements change.

3.2 Prototyping Model

Prototyping Model

How it works: Build a quick **prototype** (mock-up) to test requirements and get user feedback **early**. Refine based on feedback.

When to use:

- Requirements are **unclear or evolving**
- Need to **validate feasibility** early

Strengths: Early user feedback, reduces misunderstanding.

Weaknesses: Prototype may be mistaken for the final product; can lead to scope creep.

3.3 Evolutionary / Incremental Model

Incremental Model

How it works: Break the system into **smaller pieces (increments)**. Develop and deliver each increment one at a time. Each increment adds functionality.

When to use:

- When you want to deliver **partial functionality early**
- Requirements may **evolve** over time

Strengths: Partial working software delivered early; easier to manage.
Weaknesses: Requires good planning to define increments.

3.4 Spiral Model

Spiral Model

How it works: A **risk-driven meta-model** that combines elements of waterfall and prototyping. Each **spiral** (iteration) goes through: Planning → Risk Analysis → Engineering → Evaluation.

When to use:

- Large, **high-risk** projects
- Requirements are complex and may change

Strengths: Explicit risk management; flexible.

Weaknesses: Complex, expensive; requires risk-assessment expertise.

3.5 Comparison of Process Models

Model	Requirements	Risk Level	Flexibility
Waterfall	Stable, well-known	Low	Low
Prototyping	Unclear, evolving	Medium	Medium
Incremental	Partially known	Medium	High
Spiral	Complex, changing	High	Very High
Agile (Scrum)	Evolving, user-driven	Medium–High	Very High

3.6 The Cone of Uncertainty

Cone of Uncertainty

At the **start** of a project, estimates can be off by a factor of **4×** (over or under). As the project progresses and more information is known, estimates and actuals **converge**. This is why shorter delivery cycles and iterative development help — they reduce uncertainty faster.

Technical Debt

Technical debt is the cost of implementing **quick fixes or workarounds** that will need to be revisited later. It accumulates over time and can significantly impact the quality, maintainability, and performance of the software. **Always** try to minimize technical debt!

4 Design Principles

EXAM ALERT — From Quiz 1 Answer Key

The professor tested design principles heavily in Quiz 1. The **Open-Closed Principle** and **Refactoring** were worth 1.5 marks. **SOLID** was accepted as an alternative answer. Know these very well!

4.1 SOLID Principles

SOLID — Five Design Principles

SOLID is a set of five design principles that help create **maintainable, flexible, and scalable** software:

Letter	Principle	Meaning
S	Single Responsibility	A class should have only one reason to change (one job).
O	Open-Closed	Software entities should be open for extension, closed for modification .
L	Liskov Substitution	Subtypes must be substitutable for their base types without breaking the program.
I	Interface Segregation	Clients should not be forced to depend on interfaces they don't use .
D	Dependency Inversion	Depend on abstractions , not on concrete implementations.

4.2 Open-Closed Principle (OCP) — In Detail

Open-Closed Principle (OCP) — CRITICAL FOR EXAM

“Software entities (classes, modules, functions) should be open for extension but closed for modification.”

- **Open for Extension:** You CAN add new functionality by **extending** existing components (e.g., adding new subclasses, implementing new interfaces).
- **Closed for Modification:** You should NOT need to **change existing, working code** to add new features.

Why it matters: When you modify existing code, you risk **breaking working functionality**. By extending instead of modifying, you keep the existing codebase stable.

Example: Instead of modifying a `PaymentProcessor` class every time a new payment method is added, create an abstract `Payment` interface and add new classes like `CreditCardPayment`, `UPIPayment`, etc.

4.3 GRASP Principles

GRASP — General Responsibility Assignment Software Patterns

GRASP provides guidelines for assigning **responsibilities** to classes in object-oriented design:

- **Information Expert:** Assign responsibility to the class that has the **information needed** to fulfill it
- **Creator:** Assign class B the responsibility to create an instance of class A if B contains, aggregates, or closely uses A
- **Low Coupling:** Minimize **dependencies** between classes
- **High Cohesion:** Keep related responsibilities **together** in one class
- **Controller:** Assign the responsibility of handling system events to a **controller class**

4.4 Coupling & Cohesion

Concept	Goal	Description
Coupling	Low (loose)	Degree of interdependence between modules. Low coupling = modules work independently.
Cohesion	High	Degree to which elements in a module belong together . High cohesion = focused, single-purpose modules.

Remember

Good design = **Low Coupling + High Cohesion**. Tightly coupled systems are **rigid** and **fragile** — changing one thing breaks other things (this was explicitly tested in Quiz 1 Set B!).

4.5 Types of Software Maintenance

Four Types of Software Maintenance

Type	Description
Corrective Maintenance	Fixing bugs and defects found after release.
Adaptive Maintenance	Modifying software to work in a changed environment (new OS, new hardware, new regulations).
Perfective Maintenance	Enhancing existing features and improving performance , usability, or efficiency based on user feedback.
Preventive Maintenance	Making changes to prevent future problems . Improving maintainability, refactoring, updating documentation.

Quiz 1 Crossword: PERFECTIVE MAINTENANCE

“Perfective Maintenance” was a crossword answer. Remember: it’s about **enhancing** features carefully, not just fixing bugs. Also note “Preventive Maintenance” was mentioned in Set C answers — stopping errors **before** they occur.

5 Agile Development

5.1 The Agile Manifesto (2001)

The Agile Manifesto — Four Core Values

We value:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

*That is, while there is value in the items on the right, we value the items on the left **more**.*

5.2 12 Agile Principles (Key Ones)

- Highest priority: **satisfy the customer** through early and continuous delivery of valuable software
- Welcome **changing requirements**, even late in development
- Deliver **working software frequently** (weeks rather than months)
- Business people and developers must **work together daily**
- Build projects around **motivated individuals**
- The most efficient method of conveying information is **face-to-face conversation**
- **Working software** is the primary measure of progress
- Promote **sustainable development** — constant pace
- Continuous attention to **technical excellence** and good design
- **Simplicity** — maximize the amount of work NOT done
- The best architectures emerge from **self-organizing teams**
- Team regularly **reflects** on how to become more effective

5.3 Scrum Framework

Scrum — The Most Popular Agile Framework

Key Roles:

- **Product Owner:** Defines features, prioritizes backlog
- **Scrum Master:** Facilitates the process, removes obstacles
- **Development Team:** Self-organizing, cross-functional (3–9 people)

Key Ceremonies:

- **Sprint Planning:** Plan what to do in the next sprint (2–4 weeks)
- **Daily Standup:** 15-min meeting — What did I do? What will I do? Any blockers?
- **Sprint Review:** Demo working software to stakeholders
- **Sprint Retrospective:** Reflect and improve for next sprint

Key Artifacts:

- **Product Backlog:** Ordered list of everything needed in the product
- **Sprint Backlog:** Items selected for the current sprint
- **Increment:** The working software delivered at the end of a sprint

5.4 User Stories

User Story

A **natural-language description** of a software feature from the **user’s perspective**.

Template: “As a [type of user], I want [some goal] so that [some reason].”

Example: “As a *customer*, I want to *search for books by title* so that *I can find the book I want to buy*.”

5.5 Key Agile Practices

Practice	Description
Refactoring	Restructuring existing code to improve readability, maintainability, and efficiency without changing its behavior .
Test-Driven Development (TDD)	Write automated tests first , then write the code to make the tests pass. Cycle: Red → Green → Refactor.
Continuous Integration (CI)	Integrate code changes into a central repository frequently (multiple times a day). Automated builds and tests run on each integration.
Continuous Deployment (CD)	Automatically deploy validated code changes to production.
Pair Programming	Two developers work together at one workstation. One writes code (driver), the other reviews (navigator).

5.6 DevOps

DevOps

DevOps = **D**evelopment + **O**perations. It is a set of practices that combines software development and IT operations to **shorten the development lifecycle** and deliver software **continuously** with high quality. Key goal: minimize variability of estimates and improve project outcomes.

5.7 Agile Process Spectrum

End	Description
Plan-Driven (Rigid)	Waterfall: everything planned upfront, sequential
Middle Ground	Spiral, Incremental: some flexibility
Agile (Flexible)	Scrum, Kanban, XP: iterative, adaptive, respond to change

6 Project Estimation & Scheduling

6.1 The Iron Triangle of Project Management

Three Constraints for Project Success

A project is considered successful when it is delivered:

1. **On Time** — Meeting the deadline
2. **Within Budget** — Not exceeding the allocated cost
3. **According to Specifications** — Meeting all requirements

Only about 10–20% of projects achieve all three!

6.2 Function Point Analysis (FPA)

Function Point Analysis

A method of estimating the **size** of a software project by measuring its **functionality**. It categorizes functions into five types:

Function Type	Description
External Inputs (EI)	Data entering the system (e.g., forms, files)
External Outputs (EO)	Data leaving the system (e.g., reports, screens)
External Inquiries (EQ)	Input/output combinations (e.g., search queries)
Internal Logical Files (ILF)	Data maintained inside the system (e.g., database tables)
External Interface Files (EIF)	Data referenced from external systems

Steps:

1. Identify all functions in each category
2. Assign **complexity weights** (Simple, Average, Complex) to each
3. Calculate **Unadjusted Function Points (UFP)** = $\sum(\text{count} \times \text{weight})$
4. Apply **Value Adjustment Factor (VAF)** based on 14 general system characteristics
5. **Adjusted Function Points** = $\text{UFP} \times \text{VAF}$
6. **Effort** = $\text{Adjusted FP} \times \text{Effort per FP}$

FPA Example: Online Bookstore

Function	Type	Complexity	Weight	FP
User Registration	EI	Average	4	4
Product Listing	EO	Complex	7	7
Search Books	EQ	Average	4	4
Book Database	ILF	Complex	15	15
Payment Gateway	EIF	Simple	5	5
Total UFP				35

6.3 Value Adjustment Factor (VAF)

VAF Calculation

Based on **14 General System Characteristics (GSC)**, each rated 0–5:

- | | |
|--------------------------------|-----------------------|
| 1. Data communications | 8. Online update |
| 2. Distributed data processing | 9. Complex processing |
| 3. Performance | 10. Reusability |
| 4. Heavily used configuration | 11. Installation ease |
| 5. Transaction rate | 12. Operational ease |
| 6. Online data entry | 13. Multiple sites |
| 7. End-user efficiency | 14. Facilitate change |

$$\text{VAF} = 0.65 + 0.01 \times \sum_{i=1}^{14} \text{GSC}_i$$

The VAF ranges from **0.65** (when all GSCs = 0) to **1.35** (when all GSCs = 5).

6.4 Work Breakdown Structure (WBS)

Work Breakdown Structure

A **hierarchical decomposition** of the total scope of work into smaller, manageable tasks. Each level breaks down the work further until tasks are small enough to be estimated and assigned.

Example:

- Project: Online Bookstore
 - Phase: Design
 - * Task: Design database schema
 - * Task: Design UI wireframes
 - Phase: Implementation

- * Task: Implement user registration
- * Task: Implement search functionality

6.5 Gantt Charts & Critical Path

Gantt Chart

A **bar chart** that shows the project schedule visually. Each bar represents a task, showing its **start date**, **end date**, **duration**, and **dependencies** with other tasks.

Critical Path

The **longest sequence of dependent tasks** that determines the **minimum project duration**. Any delay in the critical path delays the **entire project**. Non-critical tasks have **slack time** (can be delayed without affecting the project end date).

Slack Time (Float)

Slack time = the amount of time a task can be **delayed** without delaying the project's end date.

$$\text{Slack Time} = \text{Latest Start} - \text{Earliest Start}$$

Key Rules:

- Tasks **on the critical path** have **zero slack time**
- Tasks **NOT on the critical path** have **positive slack time**

EXAM-STYLE Worked Example: Critical Path & Slack Time

This is modeled directly on Quiz 1, Question 2.

Consider a project with the following tasks and dependencies:

Task	Duration (hrs)	Task	Duration (hrs)
A	15	F	20
B	20	G	25
C	15	H	10
D	20	I	7
E	28		

Dependencies: A→B, A→C, B→E, C→D, E→F, D→G, F→G, G→I, F→H

Step 1: Identify all paths and their durations:

- Path 1: A → B → E → F → G → I = 15+20+28+20+25+7 = **115 hrs**
- Path 2: A → C → D → G → I = 15+15+20+25+7 = **82 hrs**
- Path 3: A → B → E → F → H = 15+20+28+20+10 = **93 hrs**

Step 2: Critical Path = the **longest path** = A → B → E → F → G → I = **115 hours**

Step 3: Slack Time for D:

- Critical path takes 115 hours total
- Path $A \rightarrow C \rightarrow D$ merges at G. After D, the path is $D \rightarrow G \rightarrow I = 25 + 7 = 32$ hrs
- Path from A to just before G via C,D: $A + C + D = 15 + 15 + 20 = 50$ hrs
- Available time from start to G via this path: $115 - 32 = 83$ hrs
- Slack for entire sub-path $A \rightarrow C \rightarrow D$: $83 - 50 = 33$ hrs BUT A is shared (on critical path), so:
- Available time for D = Total critical path – (A+C before D) – (G+I after D) = $115 - 30 - 32 = 53$
- D takes 20 hrs. Slack for D = $53 - 20 = 33 - 20 = 13$ hours (using the path comparison method)

Step 4: Slack Time for E:

- E is on the critical path, so Slack Time = 0 hours

6.6 Earned Value Management (EVM)

Earned Value Management

A method for measuring project progress by comparing:

- **Planned Value (PV):** The value of work planned to be done by now
- **Earned Value (EV):** The value of work actually completed by now
- **Actual Cost (AC):** What has actually been spent so far

Key Metrics:

- **Schedule Variance** = $EV - PV$ (positive = ahead; negative = behind)
- **Cost Variance** = $EV - AC$ (positive = under budget; negative = over budget)
- **Schedule Performance Index (SPI)** = EV / PV
- **Cost Performance Index (CPI)** = EV / AC

7 Requirements Engineering

What is Requirements Engineering?

The process of **gathering, analyzing, documenting, and maintaining** the requirements of a software system. It ensures the software meets the needs and expectations of its stakeholders. **Poor requirements gathering is the #1 reason projects fail!**

7.1 Functional vs. Non-Functional Requirements

Aspect	Functional Requirements	Non-Functional Requirements
Definition	What the system should do	How the system should behave
Focus	Specific functions/features	Quality attributes & constraints
Examples	“User can log in with username & password” “User can search for books by title” “System processes credit card payments”	“System responds within 2 seconds” “System supports 1000 concurrent users” “Data encrypted with AES-256”

7.2 Requirements Gathering Techniques

Technique	Description
Interviews	One-on-one conversations with stakeholders
Questionnaires/Surveys	Structured forms distributed to many stakeholders
Observation	Watching users perform their tasks in their environment
Brainstorming	Group sessions to generate ideas
Prototyping	Building a mock-up to validate requirements
Use Case Analysis	Modeling user interactions with the system
Workshops	Collaborative sessions with stakeholders and developers

7.3 Qualities of Good Requirements

A Good Requirement Must Be:

Quality	Meaning
Correct	Accurately reflects the stakeholder’s needs
Complete	Covers all necessary aspects; nothing is missing
Unambiguous	Can only be interpreted one way
Consistent	Does not contradict other requirements
Feasible	Can be implemented with available resources and technology
Verifiable	Can be tested to confirm it is met
Traceable	Can be linked back to its origin and forward to design/test
Concise	Short and to the point; no unnecessary words

7.4 Categorizing Requirements: Basic, Differentiators, Delights

Kano Model for Features

Category	Description
Basic (Must-Haves)	Essential features. Users assume these exist. Absence causes dissatisfaction, but presence doesn't increase satisfaction.
Differentiators	Features that distinguish your product from competitors. More = more satisfaction.
Delights	Unexpected features that exceed expectations. The “wow” factor.

Evolution: Over time, Delights → Differentiators → Basic. What was once a delight becomes the new basic expectation.

7.5 MoSCoW Prioritization

MoSCoW Method

A prioritization technique:

- **Must-Have:** Critical requirements; system doesn't work without them
- **Should-Have:** Important but not critical; can work around temporarily
- **Could-Have:** Nice to have; included if time/budget permits
- **Won't-Have (this time):** Acknowledged but deferred to a future release

7.6 Minimum Viable Product (MVP)

MVP

The **Minimum Viable Product** is a version of the product with the **smallest set of features** sufficient to satisfy early customers and gather feedback for future development. It reduces risk and validates market assumptions.

7.7 Verification vs. Validation

V&V — Know the Difference!

	Verification	Validation
Question	“Are we building the product right ?”	“Are we building the right product?”
Focus	Conformance to specifications	Meeting user needs
Activity	Reviews, inspections, walkthroughs	Testing with actual users

7.8 Software Requirements Specification (SRS)

SRS Document

The SRS is a **living document** that serves as a contract between developers and customers. It contains:

- Functional requirements
- Non-functional requirements
- Constraints
- Assumptions and dependencies
- Acceptance criteria

Important: The SRS may change throughout development as new information becomes available. Requirements analysis is an **iterative process**.

7.9 Data Privacy & Regulations

Data Privacy in Requirements

Software systems must comply with data privacy regulations:

- **GDPR** (General Data Protection Regulation) — Europe
- **India's Data Protection Act**
- Consider encryption, access controls, data anonymization
- These become **non-functional requirements** that must be addressed early!

8 Use Case Modeling

What is a Use Case?

A **use case** is a description of a **sequence of actions** that a system performs in response to an interaction initiated by an **actor**, resulting in a **specific outcome** of value. It describes the system's behavior from the **user's perspective** (the system is treated as a **black box**).

8.1 Actors

Actor

An **actor** is an external entity (person, system, or device) that interacts with the system.

Actor Type	Description
Primary Actor	Initiates the interaction with the system (e.g., Customer, Cashier)
Supporting (Secondary) Actor	Provides services or information needed by the system (e.g., Payment Gateway, Tax Calculator, Bank System)

Note: Actors are NOT limited to humans! They can be other systems, time-based triggers (e.g., a timer), or any external entity.

8.2 Use Case Template

Standard Use Case Template

Field	Description
Use Case Name	Short, descriptive name (verb + noun, e.g., "Withdraw Cash")
Brief Description	One-sentence summary of the use case
Type	Primary / Secondary
Importance	High / Medium / Low
Primary Actor	Who initiates the interaction
Supporting Actors	Other actors involved
Preconditions	What must be true BEFORE the use case starts
Main Flow	Step-by-step description of the normal (happy) path
Alternate Flows	Variations or exceptions to the main flow
Post-conditions	What is true AFTER the use case completes successfully
Non-functional Properties	Performance, security constraints (optional)

Example: ATM — Withdraw Cash

Use Case Name	Withdraw Cash
Primary Actor	Customer
Supporting Actor	Bank System
Precondition	Customer has a valid ATM card and the ATM is operational
Main Flow	1. Customer inserts ATM card 2. System prompts for PIN 3. Customer enters PIN 4. System verifies PIN with Bank System 5. System displays menu options 6. Customer selects “Withdraw Cash” 7. Customer enters amount 8. System checks balance with Bank System 9. System dispenses cash 10. System prints receipt
Alternate Flow	4a. PIN is incorrect → System prompts to retry (max 3 times) 8a. Insufficient funds → System displays error message
Post-condition	Customer has received cash; account balance is updated

8.3 Use Case Relationships

Relationship	Description
Include (⟨⟨include⟩⟩)	Use case A always uses use case B. B is a mandatory sub-behavior. Example: “Process Payment” includes “Validate Card”.
Extend (⟨⟨extend⟩⟩)	Use case B optionally extends use case A. B adds behavior under certain conditions. Example: “Checkout” may be extended by “Apply Coupon”.
Generalization	Use case B is a specialized version of use case A. Example: “Pay by Credit Card” and “Pay by PayPal” are specializations of “Make Payment”.

8.4 Granularity of Use Cases

Finding the Right Level of Detail

- Too coarse: “Use the system” — this is useless
- Too fine: “Click the button” — this is too detailed
- **Right level:** Each use case should represent a **complete, meaningful interaction** that delivers value to an actor
- Think of it as: “What can the actor accomplish in one sitting?”

9 Conceptual Modeling & UML

What is a Conceptual Model?

A **conceptual model** (also called a **domain model**) is a high-level representation of the **key concepts, attributes, and relationships** in a system. It is created by analyzing the use case model and identifies the “nouns” in the system.

It helps bridge the gap between **requirements** (problem space) and **design** (solution space).

9.1 From Use Cases to Conceptual Model

Step-by-Step Process

1. Read through all use case descriptions
2. **Extract nouns and noun phrases** — these are candidate **conceptual classes**
3. Filter out duplicates, irrelevant terms, and attributes
4. Identify **attributes** for each class (primitive data types: string, int, date, etc.)
5. Identify **relationships (associations)** between classes
6. Define **multiplicity** for each relationship
7. Draw the conceptual model using **UML class diagram notation**

9.2 UML Class Diagram Notation

UML (Unified Modeling Language)

UML is the **industry-standard** modeling language for representing software systems. For conceptual models, we use **class diagrams**.

A **class box** has three **compartments**:

ClassName
– attribute1: Type
– attribute2: Type
+ method1(): ReturnType
+ method2(param: Type): ReturnType

9.3 Access Modifiers

Symbol	Modifier	Meaning
+	Public	Accessible from anywhere
–	Private	Accessible only within the class
#	Protected	Accessible within the class and its subclasses

Key Rule

Attributes should be **private** (–). Access to them should be controlled through **public methods** (getters/setters). This is called **encapsulation**.

9.4 Associations & Multiplicity

Association

An **association** represents a **relationship** between two conceptual classes.

Multiplicity defines how many instances of one class can be associated with instances of another class.

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one (optional)
* or 0..*	Zero or more
1..*	One or more
n..m	Between n and m

Example: Point-of-Sale System

Key conceptual classes and relationships:

- **Sale** — contains — **SaleLineItem** (1..*)
- **SaleLineItem** — describes — **Product** (1)
- **Sale** — paid-by — **Payment** (1)
- **Store** — stocks — **Product** (*)
- **Cashier** — initiates — **Sale** (*)

9.5 Types of Attributes

Type	Description
Primitive	Basic data types: <code>int</code> , <code>String</code> , <code>boolean</code> , <code>Date</code> , <code>double</code>
Derived	Calculated from other attributes (e.g., <code>totalPrice = quantity × unitPrice</code>)

9.6 Key Distinction: Conceptual vs. Software Classes

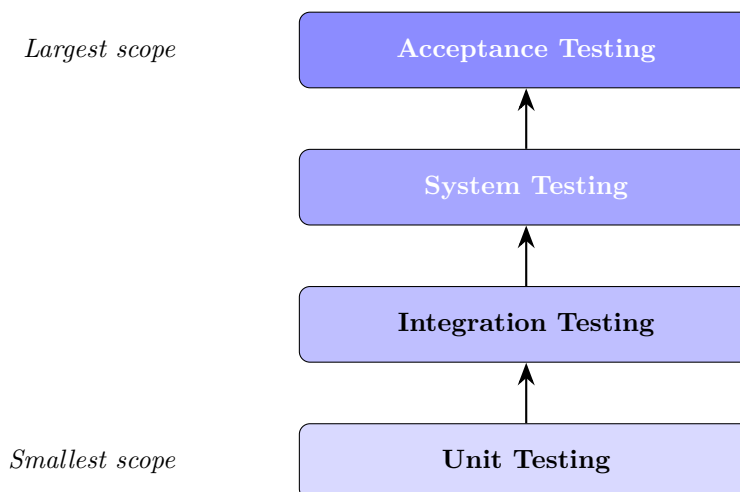
Aspect	Conceptual Class	Software Class
Level	High-level; domain concepts	Implementation-level
Purpose	Understand the problem	Solve the problem in code
Detail	No methods, just concepts	Full methods, data structures
Phase	Analysis	Design/Implementation

10 Software Testing

What is Testing?

Testing is the process of **evaluating a system or component** under **specified conditions**, **observing the results**, and making an **evaluation** of some aspect of the system. Testing is NOT just about finding bugs — it's about ensuring the system **meets requirements and standards**.

10.1 Levels of Testing



Level	Description
Unit Testing	Testing individual units of code (functions, methods, classes). Each unit is tested independently . Tests should cover all scenarios and edge cases.
Integration Testing	Testing how combined units work together . Focus is on the interfaces and communication between modules, NOT re-testing individual units.
System Testing	Testing the complete, integrated system against requirements. Performed in the real-world environment . Tests functionality, performance, security, usability.
Acceptance Testing	Testing by end-users/stakeholders to ensure the system meets their requirements and expectations. Final sign-off before release.

10.2 Integration Testing Strategies

Strategy	Description
Big Bang	Integrate all units at once and test. Simple but hard to isolate errors.
Top-Down	Start from top-level modules and integrate downward. Uses stubs for lower modules not yet ready.
Bottom-Up	Start from lowest-level modules and integrate upward. Uses drivers for upper modules not yet ready.
Sandwich / Hybrid	Combines top-down and bottom-up. Test from both ends toward the middle.

10.3 Black Box vs. White Box Testing

Aspect	Black Box Testing	White Box Testing
Knowledge	No knowledge of internal code	Full knowledge of internal code
Focus	Input/Output behavior	Internal logic & structure
Techniques	Equivalence partitioning, boundary value analysis	Statement coverage, branch coverage, path coverage
Who does it?	Usually testers	Usually developers
Also called	Functional testing	Structural testing

10.4 Black Box Testing Techniques

10.4.1 Equivalence Partitioning

Equivalence Partitioning

Divide the input data into **partitions (classes)** where all values in a partition are expected to be **treated the same way** by the system. Test **one value from each partition** instead of testing all possible values.

Example: For a field accepting ages 18–65:

- **Valid partition:** 18–65 (pick any value, e.g., 30)
- **Invalid partition 1:** < 18 (e.g., 10)
- **Invalid partition 2:** > 65 (e.g., 80)

10.4.2 Boundary Value Analysis

Boundary Value Analysis (BVA)

Test at the **boundaries (edges)** of input ranges, because errors are most likely to occur there.

Example: For the range 18–65, test:

- **Just below minimum:** 17
- **At minimum:** 18
- **Just above minimum:** 19

- Just below maximum: 64
- At maximum: 65
- Just above maximum: 66

10.4.3 State Transition Testing

State Transition Testing

Tests the system's behavior as it **transitions between states**. Useful for systems with finite states (e.g., login attempts, ATM states).

Example: ATM PIN entry: Idle $\xrightarrow{\text{insert card}}$ Waiting for PIN $\xrightarrow{\text{correct PIN}}$ Authenticated
 $\xrightarrow{\text{wrong PIN (3rd time)}}$ Card Blocked

10.5 White Box Testing Techniques

Coverage Type	Description
Statement Coverage	Every statement in the code is executed at least once
Branch/Decision Coverage	Every branch (true/false) of each decision point is executed
Condition Coverage	Every individual condition in every decision is tested as both true and false
Path Coverage	Every possible execution path through the code is tested (most thorough but often impractical)

10.6 Test Case

Test Case

A **test case** is a set of:

- **Preconditions:** What must be true before the test
- **Inputs:** The data fed to the system
- **Expected Results:** What the system should produce

Test cases are developed to drive the execution of a test item and to meet specific test objectives.

10.7 Types of Acceptance Testing

Type	Description
Alpha Testing	Done in-house by the development team or internal users
Beta Testing	Released to a limited external audience for real-world feedback
Crowd Testing	Large, diverse group of external testers
Compliance Testing	Ensures the system meets regulatory requirements and standards

10.8 Key Testing Principles

Seven Testing Principles to Remember

1. **Testing shows presence of defects**, not their absence
2. **Exhaustive testing is impossible** (too many combinations)
3. **Early testing** saves time and money
4. **Defects cluster together** (Pareto principle: 80% of bugs in 20% of code)
5. **Pesticide paradox**: Running same tests repeatedly won't find new bugs
6. **Testing is context-dependent** (different for safety-critical vs. e-commerce)
7. **Absence-of-errors fallacy**: Bug-free software is useless if it doesn't meet requirements

10.9 Testing Metrics

Metric	Description
Code Coverage	Percentage of code executed during testing
Defect Density	Number of defects per unit of code (e.g., per 1000 LOC)
Defect Detection Rate	Percentage of defects found before release
Test Pass Rate	Percentage of test cases that pass successfully

10.10 Qualities of a Good Tester

Qualities of a Good Tester

- Independent
- Curious
- Patient
- Courageous
- Detail-oriented
- Good communicator
- Good at documentation
- Systematic thinker

10.11 Regression Testing & Maintenance Testing

Regression Testing

Re-testing **existing functionality** after changes/updates to ensure that previously working features **still work correctly**. Run after every code change.

Maintenance Testing

Testing performed **after release** to ensure the system continues to meet requirements as it is updated, modified, and repaired over its lifetime.

11 Exam Traps & Warnings (From Quiz 1 Answer Key)

CRITICAL: Agile is NOT a Development Model!

From the Quiz 1 answer key: “No marks will be awarded for Agile Dev Model. Agile is not a dev model, but rather a **philosophy** (read the agile manifesto). If someone has written Scrum, they get 0. Scrum is a **process** and not a model.”

What the professor wants you to understand:

Term	What it IS
Agile	A philosophy/mindset (the Agile Manifesto has 4 values and 12 principles)
Scrum	A process framework (with sprints, roles, ceremonies)
Waterfall	A development model (sequential phases)
Prototyping	A development model (build prototype → refine)
Evolutionary / Incremental	A development model (build incrementally)
Spiral	A development model / meta-model (risk-driven iterations)

If asked “**which development model?**”, answer with Prototyping, Evolutionary, Incremental, or Spiral. **NEVER** say “Agile” or “Scrum” as a development model!

Quiz 1 Scoring Pattern — What Gets Full Marks

The professor’s scenario questions require you to **name the concept AND explain why**. Partial marks are given:

- Naming the correct concept = **half marks**
- Explaining how/why it applies = **other half**
- **Example:** “The team should use the critical path method (0.5 marks) to identify tasks that directly affect project completion. Understanding slack time (0.5 marks) allows non-critical tasks to be delayed without impacting the timeline.”

The “Magic Keywords” from Quiz 1 Answers

These concept pairs appeared repeatedly in the answer key. The professor expects you to connect them:

Question Theme	Expected Keywords
Development model choice	Prototyping + Evolutionary design
Code quality over time	Refactoring + Open-Closed Principle (or SOLID)
Testing & integration	Continuous Integration + Regression Testing
Scheduling / deadlines	Critical Path Method + Slack Time
Workflow improvements	Process Optimization + DevOps
Design flexibility	Low Coupling + High Cohesion (or SOLID/GRASP)
Maintenance practices	Perfective + Preventive maintenance

12 Crossword Vocabulary — Key Terms & Definitions

Why This Section Exists

Quiz 1 had a **14-mark crossword puzzle** (Question 3) that tested **key vocabulary**. The professor is very likely to include a similar section in the final exam. **Memorize these terms and their definitions!**

12.1 Terms from Quiz 1 Crossword (All Sets)

Term	Definition / Clue
Regression Testing	Re-testing existing functionality after changes to ensure nothing is broken
Critical Path Method	Technique to identify the longest sequence of dependent tasks determining minimum project duration
Continuous Integration	Practice of frequently integrating code changes with automated builds and tests
Slack Time	Time a non-critical task can be delayed without affecting the project end date
Refactor	Restructuring existing code without changing its external behavior
Decomposition	Breaking down a complex system/task into smaller, manageable parts (WBS)
Prototyping Model	Development model that builds a quick mock-up to validate requirements early
Open-Closed Principle	Software entities should be open for extension but closed for modification
Perfective Maintenance	Enhancing existing features and improving performance based on user feedback
Gantt Chart	Bar chart showing project schedule with task durations, start/end dates, and dependencies
Continuous Improvement	Ongoing effort to improve products, services, or processes (PDCA cycle)
Backlog	Ordered list of everything needed in a product (Product Backlog in Scrum)
Alpha Testing	In-house testing by the development team or internal users before external release
DevOps	Combination of Development + Operations for faster, continuous, quality delivery

12.2 Additional Likely Crossword Terms (New Topics Since Quiz 1)

Since Quiz 2 covers **requirements, use cases, conceptual modeling, and testing** (topics taught after Quiz 1), expect these terms:

Term	Definition / Clue
Use Case	Sequence of actions a system performs in response to an actor's interaction
Primary Actor	External entity that initiates an interaction with the system
Precondition	What must be true before a use case can begin
Postcondition	What is true after a use case completes successfully
Include Relationship	Mandatory sub-behavior always used by a use case
Extend Relationship	Optional behavior that adds to a use case under certain conditions
Encapsulation	Hiding internal state; making attributes private with public getters/setters
Multiplicity	Constraint on how many instances can participate in a relationship
Domain Model	High-level representation of key concepts, attributes, and relationships
Equivalence Partitioning	Dividing inputs into equivalent classes and testing one from each
Boundary Value Analysis	Testing at the edges/boundaries of input ranges
State Transition Testing	Testing system behavior as it transitions between states
Unit Testing	Testing individual functions/methods/classes independently
Integration Testing	Testing how combined modules work together at their interfaces
System Testing	Testing the complete integrated system against requirements
Acceptance Testing	End-user testing to validate the system meets business requirements
Stub	Dummy module used in top-down integration to simulate lower modules
Driver	Dummy module used in bottom-up integration to simulate upper modules
MoSCoW	Prioritization: Must-have, Should-have, Could-have, Won't-have
SRS	Software Requirements Specification — the living requirements document
Verification	"Are we building the product right?" — conformance to specs
Validation	"Are we building the right product?" — meeting user needs
MVP	Minimum Viable Product — smallest feature set to satisfy early customers
Traceability	Linking requirements to their origin and forward to design/test
Pesticide Paradox	Running same tests repeatedly won't find new bugs
Corrective Maintenance	Fixing bugs and defects found after release
Adaptive Maintenance	Modifying software to work in a changed environment
Preventive Maintenance	Making changes to prevent future problems
Coupling	Degree of interdependence between modules (want LOW)
Cohesion	Degree to which elements in a module belong together (want HIGH)

13 Scenario-Based Practice Questions (Quiz 1 Style)

How the Professor Designs Questions

Based on Quiz 1, the professor gives a **real-world scenario** and asks you to apply concepts from class. Each sub-question targets a specific topic area. You must **name the concept AND explain how it applies**.

13.1 Practice Scenario 1: Hospital Management System

Scenario: Hospital Management System (5 marks)

A hospital wants to build a new software system to manage patient records, appointments, billing, and pharmacy. Requirements are partially known and will evolve as doctors and staff provide feedback. The system must integrate with existing lab equipment and insurance providers. The team is small (6 developers) and the deadline is 8 months.

- (a) **(1.5 marks)** Which development model should the team adopt? Explain why.

Model Answer: The team should adopt the **Prototyping model combined with Evolutionary/Incremental design**. Since requirements are partially known and will evolve with feedback from doctors, prototyping allows early validation. Incremental delivery allows partial functionality (e.g., patient records first, then billing) to be delivered and tested early.

- (b) **(1.5 marks)** How should design and code quality be managed as new modules (pharmacy, billing) are added?

Model Answer: Code quality should be maintained through **refactoring** (keeping internal structure clean as the system grows) and adherence to the **Open-Closed Principle** (new modules like pharmacy can be added by extending existing interfaces without modifying the patient records module). SOLID/GRASP principles ensure low coupling between modules.

- (c) **(1 mark)** How can testing practices reduce integration risk with external systems (lab equipment, insurance)?

Model Answer: The team should use **Continuous Integration** to frequently merge and test code against the external interfaces, combined with **Regression Testing** to ensure that adding insurance integration doesn't break existing patient record functionality.

- (d) **(1 mark)** How should the team manage the 8-month deadline?

Model Answer: Use the **Critical Path Method** to identify which tasks (e.g., database design → patient module → billing integration) directly affect the deadline. Understanding **slack time** for non-critical tasks (like UI polish) allows flexibility when requirements change mid-project.

13.2 Practice Scenario 2: E-Commerce Platform

Scenario: E-Commerce Platform (5 marks)

A startup wants to build an e-commerce platform for selling handmade crafts. The founding team has a clear vision but the exact feature set is uncertain. They want to launch quickly with basic functionality and iterate based on user feedback. The system must handle payments, product listings, search, and order tracking.

- (a) **(1.5 marks)** Recommend a development approach and justify it.

Model Answer: **Prototyping + Evolutionary model.** The uncertain feature set makes prototyping ideal for validating ideas with real users early. Evolutionary development allows the team to ship an **MVP** (Minimum Viable Product) with core features (listings + payments) and incrementally add search and order tracking based on user feedback.

- (b) **(1.5 marks)** As the platform grows from 100 to 10,000 products, how should design quality be maintained?

Model Answer: Apply **refactoring** to improve code structure as complexity grows, and follow the **Open-Closed Principle** — e.g., the search system should be extensible (add filters by category, price range, rating) without modifying the core search logic. **Low coupling** between modules (payment, search, listings) prevents cascading failures.

- (c) **(1 mark)** How should the team ensure payment integration doesn't break existing features?

Model Answer: **Continuous Integration** with automated builds after each code change, plus **Regression Testing** to verify that payment integration doesn't affect product listing or search functionality.

- (d) **(1 mark)** The team has 12 weeks. How should they plan?

Model Answer: Use **Critical Path Method** to sequence dependencies (database schema → product module → payment → order tracking) and identify the critical path. Tasks with **slack time** (like email notifications) can be deferred if needed.

13.3 Practice: Use Case & Conceptual Modeling Question

Scenario: Room Booking System (6 marks)

A university wants a room booking system where professors can book classrooms, students can book study rooms, and admins can manage room availability.

- (a) **(1 mark)** Identify the primary and supporting actors.

Answer: **Primary:** Professor, Student, Admin. **Supporting:** University Calendar System, Notification Service, Authentication System.

- (b) **(2 marks)** Write a use case for “Book a Room” using the standard template.

Answer:

- **Name:** Book a Room

- **Primary Actor:** Professor / Student
 - **Precondition:** User is logged in; rooms are available
 - **Main Flow:** 1) User selects date/time, 2) System shows available rooms, 3) User selects a room, 4) System confirms booking, 5) System sends confirmation notification
 - **Alternate Flow:** 3a) No rooms available → System suggests alternate times; 4a) Time conflict → System shows error
 - **Post-condition:** Room is booked; appears in calendar
- (c) (2 marks) Identify conceptual classes, their key attributes, and relationships with multiplicity.

Answer:

- **User** (–name, –email, –role) — makes — **Booking** (1..*)
 - **Booking** (–date, –startTime, –endTime, –status) — reserves — **Room** (1)
 - **Room** (–roomNumber, –capacity, –building, –hasProjector) — belongs-to — **Building** (1)
 - **User** 1 — makes — 0..* **Booking**
 - **Room** 1 — has — 0..* **Booking**
- (d) (1 mark) Identify one functional and one non-functional requirement.

Answer: **Functional:** “Users shall be able to search for available rooms by date, time, and capacity.” **Non-functional:** “The system shall respond to search queries within 2 seconds for up to 500 concurrent users.”

13.4 Practice: Testing Question

Testing: Password Validation (4 marks)

A system accepts passwords that must be 8–20 characters long, contain at least one uppercase letter, one digit, and one special character.

- (a) (1 mark) Apply **Equivalence Partitioning** for the length constraint.

Answer:

- **Valid:** 8–20 characters (e.g., “Pass@1234” — 9 chars)
- **Invalid 1:** < 8 characters (e.g., “Pa@1” — 4 chars)
- **Invalid 2:** > 20 characters (e.g., 25-char string)

- (b) (1 mark) Apply **Boundary Value Analysis** for the length constraint.

Answer: Test with: 7 chars (invalid), 8 chars (valid boundary), 9 chars (valid), 19 chars (valid), 20 chars (valid boundary), 21 chars (invalid)

- (c) (1 mark) Design a **State Transition** test for login with this password system (3 attempts allowed).

Answer: Idle $\xrightarrow{\text{enter password}}$ Checking $\xrightarrow{\text{correct}}$ Authenticated. Checking $\xrightarrow{\text{wrong}}$ Attempt 2 $\xrightarrow{\text{wrong}}$ Attempt 3 $\xrightarrow{\text{wrong}}$ Account Locked.

- (d) **(1 mark)** What integration testing strategy would you use if the password module depends on the database module and the authentication module?

Answer: **Bottom-up integration:** Test the database module first (lowest level), then integrate with the password module (use a **driver** to simulate the authentication module), then integrate all three.

14 Additional Practice Questions

14.1 Short Answer Questions

Q1: Definitions

1. What is Software Engineering?
2. What is the difference between Green Field and Brown Field systems?
3. Define the SDLC and name its phases.
4. What is the PDCA cycle?
5. What is the difference between Verification and Validation?
6. Define: Use Case, Actor, Conceptual Model.
7. What is the Agile Manifesto? Name its four values.
8. What is a Function Point?
9. What is the Critical Path in a project schedule?
10. What is the difference between Black Box and White Box testing?
11. What is the Open-Closed Principle? Give an example.
12. What are the four types of software maintenance?
13. What is the difference between a Stub and a Driver?
14. What is the Pesticide Paradox in testing?

14.2 Comparison Questions

Q2: Compare and Contrast

1. Compare Waterfall vs. Agile process models
2. Compare Functional vs. Non-functional requirements
3. Compare Unit, Integration, System, and Acceptance Testing
4. Compare Primary Actors vs. Supporting Actors
5. Compare Include vs. Extend relationships in Use Cases
6. Compare Big Bang vs. Top-Down vs. Bottom-Up integration testing
7. Compare Equivalence Partitioning vs. Boundary Value Analysis
8. Compare Conceptual Classes vs. Software Classes
9. Compare Corrective vs. Perfective vs. Preventive Maintenance
10. Compare Coupling vs. Cohesion (what is desired?)

14.3 Multiple Choice Quick Review

Q3: MCQ Practice

1. The first step in SDLC is:
A) Coding B) Design C) **Feasibility Study** D) Testing
2. The #1 reason for software project failure is:
A) Bad coding B) **Poor requirements** C) No testing D) Bad UI
3. Which is NOT an Agile value?
A) Working software B) Customer collaboration C) **Comprehensive documentation** D) Responding to change
4. Maintenance typically takes what percentage of total effort?
A) 20% B) 40% C) 60% D) **70–80%**
5. Which testing technique tests at the boundaries of input ranges?
A) Equivalence Partitioning B) **Boundary Value Analysis** C) State Transition D) Path Coverage
6. A primary actor in a use case:
A) Provides services B) **Initiates the interaction** C) Is always a human D) Is optional
7. Refactoring means:
A) Adding new features B) Fixing bugs C) **Restructuring code without changing behavior** D) Writing tests
8. The Spiral model is best suited for:
A) Small projects B) **High-risk projects** C) Simple requirements D) Solo developers
9. In Earned Value Management, $CPI = EV/AC$. If $CPI > 1$:
A) Over budget B) **Under budget** C) On schedule D) Behind schedule
10. Which multiplicity means “one or more”?
A) 0..1 B) * C) **1..*** D) 0..*
11. Agile is best described as:
A) A development model B) A process framework C) **A philosophy/mindset** D) A testing technique
12. The Open-Closed Principle states that software should be:
A) Open for modification, closed for extension B) **Open for extension, closed for modification** C) Open for testing, closed for deployment D) Open source and closed source
13. Perfective maintenance is about:
A) Fixing bugs B) **Enhancing features and performance** C) Adapting to new OS D) Preventing future errors
14. In top-down integration testing, what replaces lower-level modules?
A) Drivers B) **Stubs** C) Mock objects D) Test harnesses

15. A task on the critical path has slack time of:
A) Undetermined B) Maximum **C) Zero** D) Depends on project size
16. Which of these is a CORRECT development model for uncertain requirements?
A) Waterfall **B) Prototyping** C) Agile D) Scrum

15 Quick Reference Cheat Sheet

SDLC Phases (In Order)

Feasibility → Requirements → Design → Implementation → Testing → Deployment → Maintenance

Remember: Friendly Robots Don't Ignore Testing During Maintenance

Agile Manifesto (4 Values) — AGILE IS A PHILOSOPHY!

1. **Individuals & interactions** > processes & tools
2. **Working software** > comprehensive documentation
3. **Customer collaboration** > contract negotiation
4. **Responding to change** > following a plan

Testing Levels

Unit → Integration → System → Acceptance

Remember: U Investigate Software Always

Good Requirements = CCUCFVT

Correct, Complete, Unambiguous, Consistent, Feasible, Verifiable, Traceable

Design Principles Cheat Sheet

- **SOLID** = Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
- **OCP** = Open for Extension, Closed for Modification
- **Goal:** Low Coupling + High Cohesion
- **Maintenance Types:** Corrective (bugs), Adaptive (environment), Perfective (enhance), Preventive (future-proof)

Key Numbers to Remember

Fact	Number
Lines of code moved to production per day (avg)	25 lines
Maintenance effort	70–80% of total
Development effort	20–30% of total
Project success rate	Only 10–20%
Mahindra XEV 9e lines of code	75 million
PDCA steps	Plan, Do, Check, Act
MoSCoW categories	Must, Should, Could, Won't
VAF range	0.65 to 1.35

V&V Quick Reminder

- **Verification:** “Are we building the product **right**?” (process-oriented)
- **Validation:** “Are we building the **right** product?” (product-oriented)

Quiz 1 Keyword Pairs — MEMORIZE THESE

If they ask about...	Answer with...
Development model	Prototyping + Evolutionary
Code quality	Refactoring + Open-Closed Principle
Testing practices	CI + Regression Testing
Scheduling	Critical Path + Slack Time
Workflow speed	Process Optimization + DevOps

Good Luck on Your Exam!

Remember: Understanding concepts
is more important than memorizing.
Think about **WHY** things are done, not just **WHAT** is done.

“Software engineering is about **people first**, not technology first.”

**Don’t say Agile is a development
model. Don’t say Scrum is a model.**
Always pair concept name + explanation for full marks!